

面向高风险 planner 行为干预的文献综述与工程建议

执行摘要

你的当前做法——在 prompt 后追加约束以影响 planner——在“低成本、零训练、可快速试错”方面有明显工程优势，但从现有文献看，它**不适合作为高风险 action 干预的主防线**。原因不是 prompt engineering 完全无效，而是它的**最坏情况性能、对 wording 的敏感性、以及在无外部反馈时的自纠错稳定性**都不足以支撑高风险场景。最直接的证据来自三条文献线索：其一，Huang 等指出，LLM 在**没有外部反馈**时往往不能可靠地自我纠正推理，甚至会在纠错后更差；其二，Liu 等系统证明，自反思结果对 prompt wording 高度敏感，某些反思 prompt 会把原本正确答案“纠错成错误答案”；其三，Cao 等提出“最坏 prompt 性能”视角后发现，同一模型在语义等价改写下表现波动很大，Llama-2-70B-chat 在其 RobustAlpacaEval 基准上最好与最坏表现可相差 45.48 个百分点，最坏点甚至降到 9.38%。这些结果共同说明：**prompt-only 干预可以提升平均表现，但很难给出在线 planner 所需的下界保证。** ¹

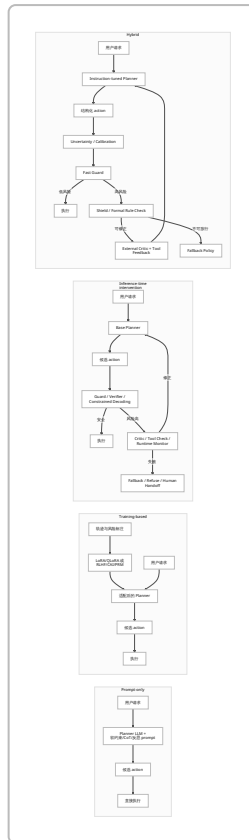
如果目标是“阻止 planner 生成高风险或明显错误 action”，更可靠的路线是把干预从“语言表层”下沉到**训练、解码、运行时监督、以及动作层规范**。文献中最有工程现实感的方案并不是单一技术，而是**混合架构**：用 instruction-tuned 或轻量 LoRA/QLoRA 适配过的 planner 生成候选计划；在动作提交前，用一个小而快的 verifier/guard model、规则/约束检查器、或概率风险监控器做**动作级 gating**；对高代价动作再触发外部 critic、工具验证、formal/symbolic shield 或 fallback policy；同时用 uncertainty / calibration 机制决定“继续执行、请求修正、降级到安全模板、或者直接拒绝”。这类做法在安全性、延迟、可扩展性之间的 trade-off，普遍优于继续堆叠 prompt 技巧。 ²

从“实时 planner 干预”的角度，近中期最值得优先投入的不是更复杂的 prompt，而是三类能力。第一类是**轻量可训练 verifier/guard**：例如 LoRA/QLoRA 微调一个 step-level 风险判别器或 Process Reward Model，用真实 planner 轨迹监督“这一步是否危险/错误/应重规划”；第二类是**动作接口约束化**：把 planner 输出约束成结构化 action schema，再叠加 constrained decoding、参数校验、符号规则或 shield；第三类是**运行时分层拦截**：低延迟 guard 先筛，风险高时再调用 critic/tool verification/runtime monitor/fallback。相比之下，纯 prompt 方法更适合做**离线 prompt search、候选计划多样化、或无训练资源时的临时补丁**，不宜单独承担高风险干预职责。 ³

问题界定与总体比较

你关心的不是一般“回答质量优化”，而是在 **planner 即将输出高风险或错误 action 时，如何在线改变其行为**。这类问题有两个关键属性。第一，它必须关注**坏例子的尾部表现**，而不是平均正确率；第二，它必须关注**动作提交前的可干预性**，而不是事后文本修订。由此，文献大体可以分成两类：一类试图通过更好的 prompt、更鲁棒的叙述方式、更多样的 reasoning scaffold 去“说服”模型；另一类则通过**参数更新、奖励建模、解码控制、外部 verifier、guard model、tool feedback、symbolic/formal shield、runtime monitor**等机制，在语言表征之外建立一层或多层控制面。前者更便宜、更通用，但脆弱；后者需要工程投入，却更接近你要的“约束 planner 行为”问题。 ⁴

下面这张示意图把四种常见架构放在同一张图里：prompt-only、training-based、inference-time intervention、以及 hybrid。图中的设计并非单篇论文原图，而是依据本综述涉及的方法抽象出的工程蓝图。相关思想分别对应 prompt scaffolding、LoRA/QLoRA 与 RLHF/CAI、SafeDecoding/Llama Guard/WildGuard/CRITIC/ShieldAgent/ProbGuard 等文献。 ⁵



下表给出一个**工程导向**的总体比较。这里的“实时干预评分”与“实现复杂度”是基于论文方法形态做出的综合判断，不是论文原始指标。评分标准偏向你的使用场景：是否能在 planner 生成 action 的在线路径上稳定、低延迟地拦截高风险步骤。

方法类别	代表论文	主要优点	主要缺点	实时干预评分	实现复杂度
软约束后缀 / 标准 CoT	CoT, Step-Back	零训练、接入快、成本低	wording 敏感、最坏情况差、无保证	2	低
Self-consistency / Prompt ensemble	Self-Consistency, Boosted Prompt Ensembles	提升平均性能、降低单样本偶然性	多采样带来高延迟，仍受 prompt 脆弱性制约	2	中
Instruction tuning	FLAN, T0	提升整体指令服从	改的是总体先验，不是在线拦截	3	中
RLHF / RLAIF / CAI	InstructGPT, Constitutional AI	全局行为塑形，安全性更稳	成本高、迁移慢、仍需运行时防线	3	高

方法类别	代表论文	主要优点	主要缺点	实时干预评分	实现复杂度
LoRA/QLoRA verifier 或 adapter	LoRA, QLoRA	数据效率高，可做轻量 behavior patch	需要标注轨迹；效果依赖数据分布	4	中
Process reward / step verifier	Let' s Verify, OmegaPRM	能对中间步骤给信用分配，最适合 planner 轨迹	训练与标注复杂；线上可能需要额外模型	4	高
Constrained decoding	SafeDecoding	在线、直接作用于生成过程	适合“局部 token/模式”风险，难处理长程语义	4	中
Guard / Safety filter	Llama Guard, WildGuard, Constitutional Classifiers	低延迟、模块化、可串联所有 planner	可能过拒；对新型攻击需持续更新	5	低到中
External critic / verifier	CRITIC, RARR	依赖外部证据，显著优于纯自反思	额外工具调用带来延迟和失败面	4	中到高
Tool-augmented planner	Toolformer, ReAct, Reflexion	把错误暴露给环境或工具，利于纠正	工具面扩大攻击面；系统更复杂	3	中到高
Symbolic / formal shield	Safe RL via Shielding, ShieldAgent	对结构化 action 可提供更强保证	需要把动作空间形式化；建设成本高	5	高
Probabilistic runtime monitor	ProbGuard	可提前预警长程风险，有阈值可调	依赖状态抽象与 traces，部署门槛较高	4	高
Model editing	ROME, MEMIT	可作持久行为/知识补丁	不适合在线单次拦截，副作用难控	1	高

表中代表工作与下文细述对应的原始来源包括 FLAN/T0、CoT、Self-Consistency、Least-to-Most、ReAct、Step-Back、APE、Active-Prompt、PromptRobust/PromptBench、RobustAlpacaEval、InstructGPT、Constitutional AI、LoRA/QLoRA、PRM、OmegaPRM、CRITIC、RARR、Toolformer、Reflexion、SafeDecoding、Llama Guard、WildGuard、Constitutional Classifiers、Safe RL via Shielding、ShieldAgent、ProbGuard、ROME 和 MEMIT。 ⁶

依赖 prompt 优化的干预文献

先说结论：这一路线在“提高平均推理能力”上非常成功，但在“稳健拦截高风险 planner action”上证据明显不足。CoT、self-consistency、least-to-most、ReAct、step-back、APE、Active-Prompt 等方法都能提升

多个 benchmark 的**平均成绩**；然而一旦把评价目标从“平均准确率”换成“最坏 prompt 下界、反思时误改正确答案的比例、对等价措辞与攻击性 paraphrase 的鲁棒性”，它们的优势会明显缩水，甚至反转。你的 prompt 后缀软约束，本质上仍属于这一类。 7

FLAN 与 T0。问题陈述是“如何让模型更好地理解自然语言指令，从而更稳定地响应 prompt”。FLAN 用 60 多个 NLP 任务做 instruction tuning，在 25 个未见任务上有 20 个超过 zero-shot GPT-3；T0 则通过多任务 prompted training 来直接诱导 zero-shot task generalization，并依赖 PromptSource 这类 prompt 资源组织任务模板。实验设置是大规模多任务 instruction 数据；关键结果是“**能显著提高 instruction following 的整体基线**”。局限是：这解决的是“模型总体会不会听指令”，不是“这一条 planner action 是否该被阻断”。代码/数据方面，FLAN 的数据生成仓库公开，T0/PromptSource 也有开源工具链。对实时 planner 来说，它们适合作为**底座能力增强**，但不构成在线干预器；复杂度中等，推理延迟几乎不变，可扩展性高，多模态迁移取决于底座是否支持多模态。 8

Chain-of-Thought。问题是如何通过显式中间推理步骤提升复杂推理能力。Wei 等发现，对足够大的模型，只需少量 CoT exemplars 就能显著提升算术、常识和符号推理；其代表性结果是，一个 540B 模型用 8 个 CoT 示例就在 GSM8K 上达到当时 SOTA，甚至超过带 verifier 的 finetuned GPT-3。局限是 CoT 提升的是“会推理”的概率，而不是“会自我约束”的可靠性；它还会增加 token 成本，并把错误中间步骤显式展开，给后续 planner 带来“看似有理”的错计划。论文页公开，未附独立官方代码仓库。用于实时 planner 干预时，复杂度低、延迟低到中等，但只能作为**生成候选 plan 的 scaffold**，不应被视为安全机制。 9

Self-Consistency。问题是 CoT 的单条 greedy 推理路径偶然性太强。方法是采样多条 reasoning paths，再对最终答案做 marginalization。实验覆盖 GSM8K、SVAMP、AQuA、StrategyQA 和 ARC-Challenge，提升分别达到 +17.9、+11.0、+12.2、+6.4 和 +3.9 个百分点。局限很直接：它把“单次偶然错误”平均掉了，但没有解决“提示词本身把模型引向错误目标”的问题；同时多采样会显著增加延迟与成本。OpenReview 页面给出论文与社区实现线索。对实时 planner 干预而言，它更适合做**离线 prompt 评估或高价值动作的二次重采样**，不适合作为全路径在线默认策略。 10

Least-to-Most Prompting。问题是 CoT 在“训练示例比测试题简单”时做不好 easy-to-hard generalization。方法先把复杂问题分解成一串更简单的子问题，再顺序求解。最醒目的结果是：在 SCAN 组合泛化任务上，GPT-3 code-davinci-002 用 14 个示例即可在所有 split 上达到至少 99% 准确率，而 CoT 只有 16%。局限在于：它更像“更好的 task decomposition prompt”，对安全干预仍然只是**间接作用**；当 planner 需要处理含隐性风险约束的 action graph 时，分解本身不等于风险控制。论文附完整 prompts。实时可行性上，复杂度低，延迟中等，可扩展到大模型，但对多模态与复杂工具状态的适配有限。 11

ReAct。问题是“推理”和“行动”分开研究，导致 planner 不会在与环境交互中利用外部信息纠偏。ReAct 将 reasoning traces 与 actions 交错生成，用 Wikipedia API 或环境反馈帮助修正 hallucination 和 error propagation；在 HotpotQA、FEVER、ALFWorld、WebShop 上表现突出，后两者相对 imitation/RL 基线分别提高 34% 和 10% 的绝对成功率。代码公开。局限是：ReAct 减少了纯语言内生推理的闭门造车，但它仍主要依赖 prompt 组织，并且**工具调用面本身会引入新的攻击面与状态同步难题**。对你的场景，ReAct 比普通 prompt-only 更接近 planner，但若没有 guard/shield，它只是“会行动的 planner”，不是“被安全约束的 planner”。 12

Step-Back Prompting。问题是模型容易被题目细节牵着走，缺乏“先抽象、后求解”的能力。方法要求模型先退一步抽取高层概念与第一性原理，再据此推理。实验在 PaLM-2L、GPT-4、Llama2-70B 上覆盖 STEM、Knowledge QA、Multi-Hop Reasoning；例如 PaLM-2L 在 MMLU 物理和化学上提升 7% 和 11%，在 TimeQA 上提升 27%，在 MuSiQue 上提升 7%。局限是抽象层 prompt 仍然是语言层技巧，遇到高风险 planner action 时，它只能提高“思考质量”，无法提供“不能做什么”的执行前保证。论文公开；未见单独官方代码仓库。实时可行性为中等：延迟增加可控，但安全性收益间接。 13

Automatic Prompt Engineer. 问题是 prompt 质量高度依赖人工手工经验。APE 把 instruction 当作要搜索的“程序”，由一个 LLM 生成候选 instruction，再由另一模型按得分函数筛选。它在 24 个 NLP 任务上大幅超过 prior baseline，并在 19/24 任务上优于或不逊于人工编写 instruction；官方仓库公开。局限是：APE 能找到“更强的 prompt”，但这并不等于“更安全或更稳健的 prompt”，尤其对 planner 安全约束，搜索目标如果仍是任务正确率，往往不会自动学到低风险行为。对实时在线使用不合适，但非常适合做**离线 prompt design baseline**。 ¹⁴

Active-Prompt. 问题是 few-shot CoT 中，人工挑选的 exemplars 往往不是最有帮助的。方法借鉴主动学习，以不确定性驱动从问题池中挑选最值得人类标注 CoT 的样本，再构建任务特定 prompt；在 8 个复杂推理任务上优于固定 exemplar 方法。局限在于它**依赖可供挑选的任务池与额外标注**，且本质仍是“更聪明地构造 prompt”，因此对高风险 action 的约束依然是软的。按 OpenReview 页面，论文公开但未在页面中给出官方仓库；实现复杂度中等，在线延迟低，适合作为 prompt dataset curate 工具，而非运行时拦截器。 ¹⁵

Prompt robustness 评测线。 PromptRobust 构造了 4,788 个跨字符、词、句子和语义层面的对抗 prompt，覆盖 8 个任务与 13 个数据集，结论是当代 LLM 对语义保持不变的 prompt 扰动并不稳健。PromptBench 则把 adversarial prompt attack、dynamic evaluation 和分析工具统一成库，代码公开。这条线的价值是：它不是“提高 prompt”的方法，而是告诉你**必须测最坏情况**。对你的问题，它的重要性高于很多华丽 prompt 技巧，因为你真正怕的是 planner 在某种措辞下突然放过高风险 action。 ¹⁶

关于 self-correction / reflection 的直接证据。 这是与你当前工作最相关的一组结论。Huang 等在 ICLR 2024 指出：如果没有外部反馈，LLM 往往不能可靠地自我纠正推理，甚至会因 self-correction 而退化。Liu 等进一步定量展示了这种脆弱性：他们在 MMLU 上从测试集随机取 1,000 题，用 Llama-3 8B 做四样本 CoT 后再做 self-reflection，发现原始答案不反思时准确率约 64.4%；文献中常见的“找错误”式反思 prompt 会把准确率降到 47.4% 到 57.8%，而改成更保守的“验证是否正确”式 prompt 后，可回升到 53.6% 到 63.5%。更关键的是，最高 false-positive correction rate 达到 40.4%，最低则只有 7.6%，说明 prompt wording 会显著改变“把正确答案误改错”的概率。作者提出的 Mixture of Prompts 在 MMLU 上将 false positive rate 压到 0.3% 到 5.3% 区间，并把最终准确率恢复到 61.5% 到 65.2%。这组结果对 planner 的含义非常直接：**prompt-only 干预最大的风险不是“没生效”，而是“在危险时没拦住，在安全时误杀正确 action”**。 ¹⁷

最坏 prompt 性能视角。 Cao 等提出 RobustAlpacaEval，强调应以“最坏 prompt 表现”而不是平均表现评估系统；他们发现 ChatGPT 与六个开源 LLM 在语义等价 case-level queries 下有巨大波动，Llama-2-70B-chat 的 best-worst gap 达 45.48 个百分点，最坏表现为 9.38%，而且现有 prompt engineering 与 prompt consistency 方法改善有限。这几乎可以看作对“只靠软约束 suffix 做高风险 planner 干预”的反证：**如果没有额外机制，你无法获得可靠的下界**。 ¹⁸

通过训练或推理机制干预模型行为的文献

这一类方法的共同特征，是把控制点从“prompt 内容”转移到**参数、奖励、logits、外部证据、动作轨迹、规则系统或运行时状态**。对高风险 planner 来说，这类方法更接近“真正的控制面”；代价是实现更复杂，需要数据、系统或形式化接口。整体上，它们比 prompt-only 更符合你的需求。 ¹⁹

InstructGPT. 问题是基础 GPT-3 并不会自然对齐用户意图。方法是先做 demonstration SFT，再用人类偏好排序训练 reward model，最后用 RLHF 进一步微调。关键结果是，1.3B 的 InstructGPT 在人工评测中比 175B GPT-3 更受偏好，同时改善 truthfulness、减少 toxic outputs。局限是：RLHF 改的是**全局行为先验**，它不保证在具体 planner action 上一定拦住风险；而且训练成本高，数据与基础设施要求高。论文公开，但公开页面未附官方训练代码/权重下载。对实时 planner 的可行性是“适合做底座，不适合作为唯一运行时安全机制”；推理延迟不增加，但迭代周期长，多模态适配取决于底座。 ²⁰

Constitutional AI. 问题是如何在减少人工标注的同时提高 harmlessness，并让模型对“为什么拒绝或修正”更透明。方法先依据一组自然语言原则让模型自我批评并修订，再用 AI preference 训练 preference

model 进行 RLAIIF。论文强调它能得到“less harmful but non-evasive”的 assistant，并指出 SL 与 RL 两阶段都能借助 CoT 提高人类评判下的透明性。局限是 constitution 仍可能覆盖不全，而且与 RLHF 一样，它更擅长全局 alignment，而不擅长动作级在线 gating。官方论文公开，未在论文页给出完整训练代码。对实时 planner，它更适合作为**行为底座与规则来源**，而不是末端拦截器。 21

LoRA 与 QLoRA。问题是全参数微调成本过高，不适合频繁给 planner 打“行为补丁”。LoRA 冻结原模型，只在若干线性层注入低秩可训练矩阵；它在 GPT-3 175B 设定下把可训练参数量降到原来的万分之一、显存降 3 倍，并且相较 adapter **没有额外推理延迟**。QLoRA 则进一步把底座量化到 4-bit，使 65B 模型能在单张 48GB GPU 上微调，且 Guanaco 在 Vicuna 上达到 ChatGPT 99.3% 的水平。两者的关键意义并不是聊天 benchmark，而是它们让你**可以很低成本地训练一个 verifier/guard/uncertainty head 或小型专用 planner adapter**。两篇论文本身都附官方代码；LoRA 更适合“单独训练一个小控制器”或“对 planner 做有监督补丁”；QLoRA 更适合资源紧张时的快速迭代。对实时 planner，这一类通常评分高，因为训练在离线、推理几乎不增或可忽略。 22

Process Reward Models。这是与你的 planner 问题最契合的一条训练路径。OpenAI 的 Let's Verify Step by Step 比较了 outcome supervision 与 process supervision，发现对 MATH 这类多步推理，step-level feedback 的 process-supervised model 明显更强，其代表模型在一个 MATH 测试子集上达到 78%，并开放了 PRM800K 这套 80 万步级标签数据。Google 的 OmegaPRM 则把过程监督自动化，通过一种 divide-and-conquer 风格的 MCTS 高效定位 CoT 中第一个错误，自动收集超过 150 万条过程监督标注，并把 Gemini Pro 在 MATH500 上从 51% 提升到 69.4%，在 GSM8K 上从 86.4% 提到 93.6%；Gemma2 27B 也有类似大幅收益。局限是 PRM 需要轨迹级/步骤级标注与单独训练，且线上若逐步打分会增加代价。对实时 planner，最值得借鉴的不是“把 PRM 当最终聊天得分器”，而是**训练一个 action-step verifier**：每一步判“是否危险/是否违背约束/是否需要 fallback”。这类方法复杂度高，但延迟可以通过小模型、蒸馏和级联控制在可部署范围。 23

CRITIC。问题是模型的内生 self-correction 不可靠，但人类会借助搜索、解释器等外部工具来核验。CRITIC 的方法是：先生成初稿，再调用相应工具评估输出的某些方面，然后依据外部反馈改写。实验覆盖自由问答、数学程序合成和 toxicity reduction，论文结论是它在这些方向上都能持续提升性能，而且研究本身强调外部反馈对 LLM 自我改进的重要性。官方实现作为 ProphetNet 仓库中的 CRITIC 模块公开。局限是：需要合适工具链，工具故障会变成系统故障，而且多一轮或多轮调用带来额外延迟。对实时 planner，它非常像“高风险动作的二级审查器”：低风险动作不走；高风险动作调用 critic + tool check，再决定放行/修正/拒绝。 24

RARR。问题是模型会生成缺乏证据支持的内容，用户又难以判断可信度。RARR 在任意文本生成模型后面加一个 attribution + revision 流水线：先自动找外部证据，再尽量少改地修订原文本，使其更有根据。论文称它只需要少量训练示例、一个 LLM 和标准 web search，就能在多个生成任务上显著改善 attribution，同时比早期 edit model 更好地保持原输出。官方实现公开。对 planner 的启示是：如果 action 需要被外部资料或工具状态“授权”，就可以把 RARR 思想转成**evidence-backed action revision**。局限仍是延迟高于 guard model，且只适合可检索、可核验的子任务。 25

Toolformer。问题是 LLM 对算术、事实查找、日历等基础能力弱，而很多高风险 planner 错误正来自错误调用这些基础能力。Toolformer 用自监督训练让模型自己学会“什么时候调用哪个 API、传什么参数、怎样把结果并回生成”。工具包括计算器、Q&A、搜索、翻译和日历，论文报告它在多个 downstream task 上显著提升 zero-shot 表现且不损伤原有语言建模能力。局限是 Toolformer 更像“让 planner 多一个正确外部世界接口”，并不能自动保证安全。官方论文公开，未见作者发布官方完整训练代码；社区有复现实现。对实时 planner，它很适合**降低因内部知识错误而引发的 planning mistake**，但必须与 guard/shield 配套。 26

Reflexion。问题是语言 agent 在环境交互中难以通过 trial-and-error 快速学会更好的策略。Reflexion 不更新权重，而是把 task feedback 写成 verbal reflection 存入 episodic memory，在后续回合中影响决策；它在 HumanEval 上达到 91% pass@1，超过文中给出的 GPT-4 80%，并在序列决策、代码、语言推理任务上都有收益。官方仓库公开。与“无外部反馈的反思”不同，Reflexion 依赖环境反馈，因此与 Huang 等批评

的“intrinsic self-correction”并不矛盾。局限是多轮试错会引入较高交互成本，不适合每个 planner action 都走完整反馈循环。它更像**可持续学习的 planner 提升框架**，而不是低延迟刹车装置。 27

SafeDecoding. 问题是即便模型已对齐，jailbreak 仍可能通过解码路径把有害 token 推到前面。SafeDecoding 的核心观察是：即使 harmful token 概率更高，安全 disclaimer token 常常仍位于 top tokens 中，因此可以在解码时提升安全 token 概率、抑制与 jailbreak 目标一致的 token 序列。论文在五个 LLM、六种 SOTA jailbreak、四个 benchmark dataset 上测试，结论是它在无明显伤害 benign helpfulness 的情况下显著降低 attack success rate 和 harmfulness，超过六种 defense；官方代码公开。局限是：它对“toxicity/unsafe response”这种 token-level 或局部语言模式很有效，但对 planner 的复杂长程错误动作，必须先把输出接口结构化，才便于把 decoding control 作用到动作层。若你的 planner action 是 JSON/函数调用/schema，SafeDecoding 类策略很值得试。 28

Llama Guard、WildGuard 与 Constitutional Classifiers. 这三类工作代表的是“**独立安全判别器/guard model**”路线。Llama Guard 是一个针对输入与输出双向做分类的 safeguard model，基于 Llama2-7B instruction tuning，在 OpenAI Moderation Evaluation 和 ToxicChat 上匹配或超过已有 moderation 工具，且 Meta 明确开放了权重。WildGuard 则进一步做成“一站式 moderation”，同时判 prompt harmfulness、response harmfulness 与 refusal；它构建了 92K 的 WildGuardMix 和 5K 的高质量测试集，在十个公开 benchmark 上相对十个开源 moderation 模型实现整体 SOTA，例如 refusal detection 最多提升 26.4%，还把 LLM 接口中的 jailbreak 成功率从 79.8% 压到 2.4%，官方代码与模型均公开。Constitutional Classifiers 则把“根据 constitution 生成合成训练数据，再训练 safeguard classifier”推向更激进的防越狱方向：作者报告超过 3,000 小时红队测试未发现能在多数目标查询上像未防护模型那样抽取细节的 universal jailbreak，并声称生产流量中的 refusal 仅绝对增加 0.38%，推理开销约 23.7%。对实时 planner，这一类是**性价比最高的第一道门**：独立、低延迟、可插拔、可做级联；不足是容易 overblock，且需要持续追踪新型攻击。 29

Uncertainty 与 calibration. 这条线对 planner 干预的价值，在于它给出“**什么时候不该继续执行**”的信号。Kadavath 等表明，模型可以对自己答案正确性的概率做一定程度的自我评估，提出了 $P(\text{True})$ 与 $P(\text{IK})$ 等概念；Kapoor 等的 Calibration-Tuning 与后续 NeurIPS 论文则进一步指出，**光靠 prompting 不足以得到好的校准**，但只要用一小批正确/错误答案做微调，就能得到泛化更好的 uncertainty estimate，而且约一千个 graded examples 就足以超过多种 baseline，且可用 LoRA 在大开源模型上高效训练。Farquhar 等的 semantic entropy 又提供了 task-agnostic 的 confabulation 检测方式，在新任务上也较稳健。对 planner，这意味着你可以训练或外挂一个**风险/不确定性层**：高不确定时，不是继续生成动作，而是触发 critic、fallback、或 human handoff。局限是 uncertainty 并不等于 safety，且 calibration 在分布外仍会漂移。 30

Shielding 与 formal/runtime governance. 传统 safe RL 文献中的 shield 思想非常适合迁移到 planner：给定 temporal logic 或显式安全规范，shield 在 learner 出手前给出安全动作集，或在其出手后直接替换违反规范的动作；它的目的就是让 agent 在学习或执行期间都不踩线。到了 LLM agent 场景，ShieldAgent 把这种思想推进到 action trajectory 级别：先从 policy documents 提取可验证规则，构成 action-based probabilistic rule circuits，再对受保护 agent 的轨迹做逻辑推断与 formal verification。论文给出一个 3K 级别的 ShieldAgent-Bench，覆盖 6 个 web 环境与 7 类风险；更细致的 ar5iv 内容显示 benchmark 中有 3,110 条不安全轨迹，ShieldAgent 在整体 guardrail accuracy 上约 90.4%，FPR 5.6%，并显著降低查询数与时间。ProbGuard 则把治理从“反应式规则触发”升到“**概率风险预警**”：它从执行 traces 学出 DTMC，对未来到达 unsafe states 的概率做 reachability 估计；在自动驾驶与 embodied household agents 两类场景中，它可提前最多 38.66 秒发出警告，并把不安全行为最多降低 65.37%，同时最多保住 80.4% 的任务完成率，且作者提供了开源实现。对你要的“实时 planner 干预”，如果 action space 能结构化，**shield + runtime monitor 是目前文献里最像真正安全控制器的东西**。代价是它们需要显式状态抽象、规则工程与运行时集成。 31

Model editing. ROME 用因果干预定位并修改 GPT 中存储事实联想的中层 MLP 模块；MEMIT 则把编辑规模扩展到数千条关联。两者都证明了“通过直接修改模型内部机制改变行为/知识”在技术上可行，而且 ROME 本身就带有强烈的 causal tracing / mediation 风格，对调试“哪些内部状态促成危险 action 倾向”很有启发。局限非常明确：这更适合做**离线持久补丁**，不适合做在线单次 action 拦截；而且行为编辑的副作用、泛化与可

撤销性都需要额外评估。对实时 planner，我会把它放在低优先级：适合修补系统性、稳定复现的已知危险偏好，不适合作为主干预路线。 32

对实时 planner 干预的结论

如果把问题精确化成“当 planner 即将发出一条可能高风险或错误的 action 时，哪类方法最可行”，答案并不是“继续优化 prompt”，而是把 **planner 的控制点前移到动作层、下沉到独立 guard，并在高风险路径上追加外部验证**。在这方面，最强的证据链是：prompt wording 会改写 false-positive/false-negative 自纠错行为，最坏 prompt 表现不可靠；而 PRM、guard models、shielding、runtime monitor 与 tool-grounded critic 则都在各自 benchmark 上提供了更接近“可控拦截”的行为。 33

就工程架构而言，最稳健的选择不是“二选一”，而是**混合式四层栈**。第一层，用 instruction tuning 或轻量 adapter 让 planner 基本听话，减少明显无效动作；第二层，在 action schema 上加 guard model、参数验证、allow/deny 规则和 constrained decoding，把绝大多数低级风险挡在毫秒级；第三层，对高风险 action 触发 external critic/tool verifier/process reward scorer；第四层，对有明确政策或长程风险的领域，用 shield 或 probabilistic runtime monitor 决定是否 rollback、fallback 或交人。这个组合恰好对应文献的优势分工：prompt 与 instruction tuning 负责可用性，LoRA/PRM 负责专项行为塑形，guard 负责低延迟拦截，critic/tool/shield 负责高风险兜底。 34

从“单项技术”的投入产出比看，我认为优先级最高的三项分别是：**独立 guard model、LoRA/QLoRA 训练的 step-level verifier/uncertainty head、以及结构化 action + runtime shield/fallback**。独立 guard model 之所以排第一，不是因为它最学术，而是因为它最容易落地到现有 planner pipeline：输入候选 action 和上下文，输出“放行/拦截/升级审查”。LoRA verifier 排第二，是因为它能学习你自己的“错误 action 分布”，比通用 moderation 更贴近业务。runtime shield 排第三，是因为它最接近“有保证的系统”，尤其在动作空间有限、政策规则清晰时价值极高。相反，继续在 prompt 后缀上堆 soft constraints，即便短期有收益，也更像是“把控制寄希望于模型自觉”。就高风险 planner 而言，这条路线应降级为辅助手段。 35

建议的实验设计与评估指标

实验设计应避免只看“最终任务是否成功”，而要显式衡量 **intervention 是否正确地改写了错误行为**。最合适的 protocol 是三层评测。第一层是**离线静态 replay**：给定历史 planner 轨迹和人工标注的“危险/错误/可放行/需回退”标签，评估 guard、verifier、PRM 与 uncertainty signals。第二层是**受控环境在线评测**：在 ALFWorld、WebShop、SafeAgentBench 或内部沙盒中，让 planner 真正执行，以观察 intervention 对 task success、unsafe action rate 与回退行为的影响。第三层是**prompt robustness/stress test**：对同一用户意图做多轮 paraphrase、角色切换、冗余上下文、攻击式包装和中英混写，测试最坏情况表现。用 RobustAlpacaEval/PromptRobust 的思想来评估 planner wording 稳健性，是必要而不是可选项。 36

数据集方面，建议至少包含四个桶。其一是**你自己的 planner production-like traces**，这是最重要的数据源，因为真正的高风险 action 往往高度 domain-specific；建议优先收集“接近放行但最终被人工纠正/拒绝”的 hard negatives。其二是**推理与自纠错数据**，如 MMLU、GSM8K、CommonSenseQA，用于量化 reflection/verification 对纯 reasoning error 的作用。其三是**安全与越狱/拒答数据**，如 WildGuardMix/WildGuardTest、OpenAI Moderation Eval、ToxicChat 一类输入输出安全数据，用于训练低延迟 guard。其四是**agent/planner 基准**，如 SafeAgentBench、ALFWorld、WebShop，以及如果你的系统是 web agent，还应加入 ShieldAgent-Bench 一类轨迹级 guardrail benchmark。 37

最关键的指标不应只包括 accuracy，而应至少包括以下九类。**False-positive correction rate**：原本正确/安全的 action 被误改或误拒的比例；Liu 等的结果说明它会直接吞噬最终准确率。**False-negative rate**：危险/错误 action 没被拦下的比例。**Correction success rate**：对原本错误 action，干预后成功修正为安全/正确动作的比例。**Hazard-weighted miss rate**：对高危类别按严重程度加权后的漏检率。**Latency**：p50/p95/p99 的额外时延；对级联系统要分 guard、critic、tool、shield 各层统计。**Throughput**：每秒可处理 action 数。

Robustness to prompt wording: 同一语义簇下 best-worst gap、worst-case accuracy、以及跨 paraphrase 的 decision consistency。**Distribution shift:** 换工具、换策略文档、换语言、换环境噪声、换任务分布后的性能衰减。**Calibration:** ECE、Brier score，或对 planner 风险分数的 risk-coverage 曲线；对开放式生成/规划问题，也可用 semantic entropy 一类信号评估“该不该继续执行”。SafeDecoding 所用 ASR 及其 helpfulness 约束也可以作为安全对抗面的补充指标。 38

在实验切分上，我建议明确做三种 OOD。其一是 **prompt OOD**：训练只见一种或少数 system/user wording，测试时用 paraphrase、冗余上下文、角色包装、攻击模板评测；其二是 **policy OOD**：训练 guard 时只见旧版规则，测试时加入未见过的策略条目、合规禁令或例外条件；其三是 **tool/state OOD**：换工具 API、换观察噪声、换环境状态抽象，测试 shield/runtime monitor 是否 still works。高风险 planner 真正会栽在这三种 OOD 上，而不是普通 benchmark 平均分。 39

工程建议与风险缓解

下面这些建议按“可立即实施且与实时 planner 干预最相关”的优先级排序。优先级不是“学术先进程度”，而是“对你现有系统的边际收益 / 实施成本 / 风险控制价值”的综合判断。其依据主要来自上文关于 prompt 脆弱性、PRM、guard model、shield 与 runtime monitor 的证据。 40

优先级	建议	立刻可做的实现	主要风险	缓解策略
P0	把“软约束后缀”降级为辅助，而不是唯一安全机制	在 planner 输出 action 前增加独立 guard 接口，至少先做放行/拒绝/升级审查三类	误以为 prompt 仍是主防线，导致尾部漏检	将所有高严重度动作强制经过 guard；把 prompt 仅留作生成质量优化
P0	训练一个小型 step-level verifier	用 LoRA/QLoRA 在 1k-10k 条真实 planner 轨迹上训练“危险/错误/需重规划”分类器	数据分布偏、标注不一致	先从高风险动作和人工纠正样本做 hard-negative bootstrap；做双人复标
P0	把 action 输出结构化	统一到 JSON / function call / schema，便于 constrained decoding、arg 校验与规则检查	结构化不完整，模型转而把风险藏进自由文本字段	对字段级做 allowlist、单位/范围校验、枚举约束和正则/AST 检查
P1	为高风险动作引入 critic + tool verification	例如检索证据、读策略文档、问状态 API，再决定是否重写 action	额外延迟与工具失败面	采用级联：只有 guard 标记高风险才触发 critic；工具失败时默认 fallback
P1	加 uncertainty / calibration gating	训练独立置信层，或在高价动作上启用 semantic entropy / risk score	校准在分布外失效	周期性重校准；对高风险域使用 conservative threshold 与人工抽检
P1	引入 fallback policy	一旦判定风险高或 verifier 不一致，切换到安全模板、只读模式、人工确认或拒绝	任务成功率下降	做 severity-aware fallback：高危严格、低危宽松；持续优化误拒

优先级	建议	立刻可做的实现	主要风险	缓解策略
P2	为规则清晰的 domain 建 shield / runtime monitor	finance、ops、web automation、企业 policy 执行等最适合；规则可先手工写，再逐步自动化	初期规则工程工作量大	先覆盖 20% 的高危规则；用日志发现高频违规后迭代扩充
P2	建立对抗与 wording 稳健性回归套件	每次更新都跑 paraphrase、越狱模板、上下文污染、系统提示冲突测试	测试集过于“教条”，无法反映真实用户输入	混合真实日志改写、人工红队与自动生成 paraphrase，持续刷新集合

特别需要强调的风险有四类。第一类是**过拦截**：guard 引入后任务成功率和用户体验可能显著下降；这在 WildGuard、Constitutional Classifiers 这类工作里都被当成部署核心指标。第二类是**欠拦截**：guard/PRM 只会学习你见过的风险形态，对新型 prompt injection、长程后果或策略组合漏洞可能失效。第三类是**延迟膨胀**：如果把 self-consistency、critic、多工具验证全部串到主路径，planner 会很快不可用。第四类是**日志与标注风险**：训练 verifier 需要真实 planner 轨迹，而这些轨迹往往包含用户数据、组织策略与工具结果。对应的缓解思路分别是：做 severity-aware thresholds 与 cascaded defense；保持持续红队和 OOD 评测；采用 fast-path/slow-path 两级结构；以及对日志做最小化采集、PII 脱敏、分级访问与安全审计。⁴¹

开放问题与局限

当前文献有一个明显空白：**真正针对“planner 在实时生成 action 时如何被可靠干预”的工作，仍少于一般聊天安全与推理优化工作**。大量 prompt 文献是在 QA、math、reasoning benchmark 上评估；大量安全文献是在文本 harmfulness 或 jailbreak 上评估；真正把“动作轨迹、工具状态、策略文档、长期后果、回退控制”连起来研究的，主要是近两年的 ShieldAgent、ProbGuard、SafeAgentBench 这类工作，而它们多数仍较新，外部复现实证与产业级长期部署证据有限。⁴²

另一个局限是，多模态与超大闭源模型下的结论并不完全对齐。LoRA/QLoRA、guard models、PRM、shielding 在开源文本模型上有较清晰的工程路径；但对于只提供 API 的黑盒多模态 frontier model，你能做的往往更多是**外插式 runtime intervention**，而不是对内部参数、表示或解码器进行深度控制。这会让系统更倾向于“外部治理层”而不是“内生行为微调”。换言之，越黑盒、越多模态，你越应该投资 guard、critic、tool verification、runtime monitor 与 fallback，而不是继续押注 prompt suffix。⁴³

综合这批文献，如果只保留一句最重要的判断，那就是：**对高风险 planner，prompt engineering 应当被视为“可用性增强层”，而不是“安全控制层”；真正可靠的干预，应建立在训练出的 verifier/guard、结构化动作约束、外部证据核验，以及运行时 shield / fallback 之上。**⁴⁴

¹ ¹⁷ ⁴⁴ <https://openreview.net/forum?id=lkmD3fKBPQ>

² ¹⁹ ²⁰ <https://arxiv.org/abs/2203.02155>

³ ²² ⁴³ <https://arxiv.org/abs/2106.09685>

⁴ ⁷ ⁹ ⁴² <https://arxiv.org/abs/2201.11903>

5 6 8 34 <https://arxiv.org/abs/2109.01652>
<https://arxiv.org/abs/2109.01652>

10 <https://openreview.net/forum?id=1PL1NIMMrw>
<https://openreview.net/forum?id=1PL1NIMMrw>

11 <https://openreview.net/forum?id=WZH7099tgfM>
<https://openreview.net/forum?id=WZH7099tgfM>

12 36 <https://arxiv.org/abs/2210.03629>
<https://arxiv.org/abs/2210.03629>

13 <https://arxiv.org/abs/2310.06117>
<https://arxiv.org/abs/2310.06117>

14 <https://arxiv.org/abs/2211.01910>
<https://arxiv.org/abs/2211.01910>

15 <https://openreview.net/forum?id=wabp68RoSP>
<https://openreview.net/forum?id=wabp68RoSP>

16 <https://arxiv.org/abs/2306.04528>
<https://arxiv.org/abs/2306.04528>

18 39 <https://arxiv.org/abs/2406.10248>
<https://arxiv.org/abs/2406.10248>

21 <https://arxiv.org/abs/2212.08073>
<https://arxiv.org/abs/2212.08073>

23 <https://arxiv.org/abs/2305.20050>
<https://arxiv.org/abs/2305.20050>

24 <https://arxiv.org/abs/2305.11738>
<https://arxiv.org/abs/2305.11738>

25 <https://arxiv.org/abs/2210.08726>
<https://arxiv.org/abs/2210.08726>

26 <https://arxiv.org/abs/2302.04761>
<https://arxiv.org/abs/2302.04761>

27 <https://arxiv.org/abs/2303.11366>
<https://arxiv.org/abs/2303.11366>

28 <https://arxiv.org/abs/2402.08983>
<https://arxiv.org/abs/2402.08983>

29 <https://arxiv.org/abs/2312.06674>
<https://arxiv.org/abs/2312.06674>

30 <https://arxiv.org/abs/2207.05221>
<https://arxiv.org/abs/2207.05221>

31 <https://arxiv.org/abs/1708.08611>
<https://arxiv.org/abs/1708.08611>

32 <https://arxiv.org/abs/2202.05262>
<https://arxiv.org/abs/2202.05262>

33 37 38 40 <https://ar5iv.org/html/2406.10400v1>

<https://ar5iv.org/html/2406.10400v1>

35 41 <https://arxiv.org/abs/2406.18495>

<https://arxiv.org/abs/2406.18495>